Chapter 8: Stochastic Vehicle Routing Problems

Vehicle Routing: Problems, Methods, and Applications, 2nd ed. (2014)

Michel Gendreau Ola Jabali Walter Rei

Lecture slides prepared from Chapter 8

Chapter Outline

Introduction: Why Stochastic VRP? I

Vehicle Routing Problems (VRPs) have been the subject of intensive research since the landmark paper of Dantzig and Ramser (1959), followed by the Clarke–Wright savings algorithm (1964) and decades of subsequent work. The vast majority of this literature treats VRP as a purely *deterministic* problem: all customer demands, locations, and travel times are assumed to be known with complete certainty at the time routes are planned.

In practice, however, these quantities are *uncertain*. A distribution company dispatches its fleet in the morning based on forecasted demands, yet the actual demand at each stop is only revealed when the vehicle arrives. A service technician plans a daily route, but some customers may cancel appointments or be unavailable. An urban courier cannot predict traffic delays precisely enough to guarantee on-time arrivals.

Introduction: Why Stochastic VRP? II

Definition — Stochastic VRP (SVRP)

A **Stochastic VRP** (SVRP) is any Vehicle Routing Problem in which one or more parameters are *random variables* whose exact values are not known at the time the route plan is constructed:

- customer **demands** D_i (how much each customer needs — random because orders vary day-to-day),
- customer **presence** Y_i (whether a customer will actually require service — uncertain because appointments may be cancelled), or
- arc **travel times** T_{ij} (how long it takes to travel from node i to node j — uncertain due to traffic, weather, or service-time variability).

The central challenge is that a route which is optimal under the *average* scenario may perform very poorly under certain realizations. Ignoring uncertainty leads to capacity violations (vehicles run out of stock mid-route), missed time-window deadlines, or costly emergency restocking trips that were not in the original plan.

Introduction: Why Stochastic VRP? III

Two fundamentally different paradigms have been developed for dealing with stochasticity in VRP, and understanding the distinction between them is essential before studying any specific model.

Paradigm 1 — A Priori Optimization. An a priori (Latin: "before the fact") solution is a fixed route plan computed *before* any random quantity is revealed. The quality of this plan is measured by its *expected cost* (average cost over all possible random outcomes). When actual demands are revealed during execution, only minor local corrections are made according to a pre-specified *recourse policy* (for example: if the vehicle runs out of capacity, return to the depot, reload, and continue).

Paradigm 2 — Reoptimization. The route is not fixed in advance. After each random revelation (e.g., the demand at a customer is observed upon arrival), the plan for all remaining customers is completely recomputed. This yields higher solution quality but requires solving a new VRP instance online at each decision point, which is computationally expensive.

Introduction: Why Stochastic VRP? IV

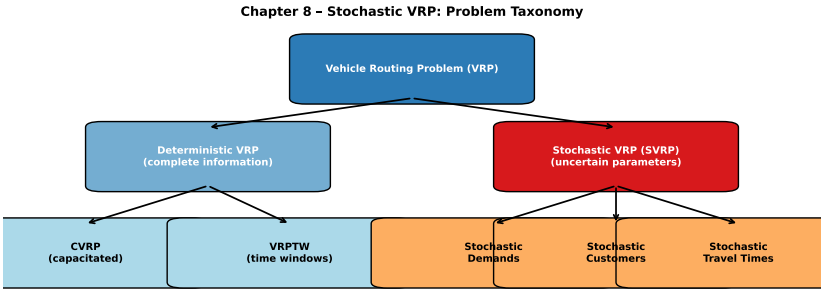


Figure: Taxonomy of VRP variants showing deterministic versus stochastic branches. The three main SVRP types (stochastic demands, customers, and travel times) correspond to different uncertain parameters.

Introduction: Why Stochastic VRP? V

The stochastic VRP literature is broadly classified into three major problem types, each defined by which parameter is uncertain:

Type 1 — VRPSD: VRP with Stochastic Demands. All customers are known and will definitely be visited, but the exact demand D_i (read: "D sub i", the load the vehicle must pick up or deliver at customer i) is a random variable revealed only upon arrival. The main risk is that the total demand along a route exceeds the vehicle capacity Q (the maximum weight or volume the vehicle can carry), forcing an unplanned return to the depot to reload.

Type 2 — VRPSC: VRP with Stochastic Customers. Each customer i is present with a known probability p_i (read: "p sub i", the probability that customer i is actually home or requires service on a given day) and absent with probability $1 - p_i$. The set of customers requiring service is revealed only at execution time.

Type 3 — VRPST: VRP with Stochastic Travel Times. Travel times T_{ij} (read: "T sub i j", the travel time from customer i to customer j) are random variables. The primary concern is time-window feasibility: the vehicle may arrive too early (and must wait, wasting time) or too late (violating a customer's service window, incurring a penalty).

Introduction: Why Stochastic VRP? VI

Stochastic VRP research began with Tillman (1969) and was advanced theoretically by Bertsimas (1992). The influential survey by Gendreau, Laporte, and Séguin (1996) remains the canonical reference, and the chapter we study here provides a comprehensive update of the field as of 2014.

Key Insight: The Core Tension

The fundamental difficulty of SVRP is that one must commit to a plan *before* the random values are known, yet the quality of the plan is judged *after* they are revealed. A plan optimised for the average scenario sacrifices performance in the tail scenarios. Good SVRP algorithms explicitly model this tension and plan for uncertainty, not just for the expectation.

A Priori Optimization: Framework and Network Formulation I

Under the a priori optimization paradigm, the goal is to find a single fixed route plan that achieves the best *expected* performance across all possible realizations of the random parameters, given a specified recourse policy that governs how minor corrections are made during execution.

Graph and Notation Setup

Let $G = (V, E)$ be a complete undirected graph where:

- $V = \{v_0, v_1, \ldots, v_n\}$ is the set of nodes. Node v_0 ("v sub zero") is the **depot** (the warehouse where vehicles start and end). Nodes $V' = \{v_1, \ldots, v_n\}$ are the **customers**.
- $E = \{(v_i, v_j) : i < j,\ i, j \in V\}$ is the edge set (the set of all possible direct connections between nodes).
- $c_{ij} \geq 0$ ("c sub i j", read "cost from i to j") is the travel cost of edge (v_i, v_j), typically distance or time.

Each customer v_i has a **random demand** D_i ("D sub i") drawn from a known probability distribution F_i. A fleet of vehicles, each with capacity Q, must serve all customers while minimising expected total travel cost.

A Priori Optimization: Framework and Network Formulation II

The random variables $D_1, D_2, \ldots, D_n$ are assumed to be *independently distributed* (each customer's demand is independent of all others), though they may have different distributions. This is a standard and realistic assumption: one customer's order size does not affect another customer's order size.

A Priori Optimization: Framework and Network Formulation III

The Restocking Recourse Policy. The most widely studied recourse policy for VRPSD is the *optimal restocking* policy. Let $\pi = (v_0, v_{i_1}, v_{i_2}, \ldots, v_{i_n}, v_0)$ be the a priori route (the planned visiting order). The vehicle follows this sequence. When it arrives at customer v_{i_k} and finds that serving v_{i_k} would cause the accumulated demand to exceed the capacity Q, the vehicle *returns to the depot* (cost $c_{v_{i_{k-1}}, v_0}$), reloads to full capacity, then returns to v_{i_k} (cost $c_{v_0, v_{i_k}}$) and continues the route. This unplanned return trip is called a **recourse trip**, and its cost is the *recourse penalty*.

A Priori Optimization: Framework and Network Formulation IV

Worked Example — Recourse Trip

Suppose $Q = 10$, route $\pi = (v_0, v_1, v_2, v_3, v_4, v_0)$, and one realization of demands is $d_1 = 4, d_2 = 5, d_3 = 3, d_4 = 2$.

Step 1: Serve v_1 (accumulated load $= 4 \leq 10$). OK.

Step 2: Serve v_2 (accumulated load $= 4 + 5 = 9 \leq 10$). OK.

Step 3: Attempt v_3: accumulated load would be $9 + 3 = 12 > Q = 10$. Recourse trip: drive from v_2 to depot (cost $c_{v_2 v_0}$), reload, drive from depot to v_3 (cost $c_{v_0 v_3}$). Now serve v_3 (load $= 3$).

Step 4: Serve v_4 (load $= 3 + 2 = 5 \leq 10$). Return to depot.

The total execution cost equals the planned route cost *plus* the recourse penalty $c_{v_2 v_0} + c_{v_0 v_3}$. Since d_1, d_2, d_3, d_4 are random, this recourse penalty is itself random, and the VRPSD minimises its *expected value*.

A Priori Optimization: Framework and Network Formulation V

The a priori SVRP for VRPSD can be cast as a stochastic integer program. We follow the formulation of Christiansen and Lysgaard (2007) and Gendreau, Laporte, and Séguin (1995, 1996).
Let $\hat{G} = (\hat{V}, \hat{A})$ be the complete directed graph on the same node set, where $\hat{A}$ contains a directed arc (i, j) for each ordered pair of distinct nodes. The decision variables are:

- $x_{ij} \in \{0, 1\}$ ("x sub i j"): equals 1 if arc (i, j) is traversed in the a priori plan, and 0 otherwise. This variable encodes the routing structure.
- $\boldsymbol{D} = (D_1, \ldots, D_n)$ ("bold D"): the random demand vector.

Network Flow Formulation — Objective and Constraints (8.1)–(8.7)

Minimise the expected total cost:

$$E[\mathcal{C}(\pi)] = \underbrace{\sum_{(i,j)\in\hat{A}} c_{ij}\, x_{ij}}_{\text{planned travel cost}} + \underbrace{\sum_{i\in V'} (c_{v_0 v_i} + c_{v_i v_0})\, E\big[R(\pi, \boldsymbol{D}, i)\big]}_{\text{expected recourse cost}} \tag{1}$$

subject to:

$$\sum_{j\in V'} x_{v_0 j} = m \tag{8.2}$$

$$\sum_{i\in V\setminus\{k\}} x_{ik} = 1 \qquad \forall k \in V' \tag{8.3}$$

$$\sum_{j\in V\setminus\{k\}} x_{kj} = 1 \qquad \forall k \in V' \tag{8.4}$$

$$x_{ij} \in \{0,1\} \qquad \forall (i,j) \in \hat{A} \tag{8.5}$$

The Integer L-Shaped Algorithm I

The **integer L-shaped algorithm** is the leading exact method for the a priori VRPSD. It was originally developed by Laporte and Louveaux (1993) for general two-stage stochastic integer programs and adapted by Gendreau, Laporte, and Séguin (1995) specifically for the VRPSD.
The algorithm belongs to the family of *Benders decomposition* methods (named after the mathematician J.F. Benders, 1962). The core idea is to split the problem into two interacting parts:

1. A **master problem** that optimises the routing structure (the binary variables x_{ij}), treating the recourse cost as a variable θ (theta, a lower-bound estimate).
2. A **subproblem** that, given a candidate route π from the master, computes the exact expected recourse cost $E[\mathcal{C}(\pi)]$ using the DP recursion described earlier, and generates *optimality cuts* that prevent the master from underestimating the recourse cost in future iterations.

The Integer L-Shaped Algorithm II

The name “L-shaped” comes from the shape of the constraint matrix when the subproblem constraints are appended to the master problem in a tableau form — it resembles a capital letter L.

The Integer L-Shaped Algorithm III

Integer L-Shaped Algorithm — Full Pseudocode

Initialisation.

Set $\theta \leftarrow -\infty$ (theta, initial lower bound on recourse cost; starts at negative infinity meaning no lower bound yet).

Set iteration counter $\ell \leftarrow 0$ (ell, current iteration number).

Step 1 (Master Problem).

Solve the LP or MIP relaxation of the master:

$$\min \sum_{(i,j)} c_{ij}\, x_{ij} + \theta \quad \text{s.t. constraints (8.2)–(8.5) + all cuts generated so far.}$$

Let (x^*, θ^*) be the optimal solution from this iteration.

Step 2 (Subproblem — Recourse Evaluation).

For each route π^* encoded by x^*, compute $E[\mathcal{C}(\pi^*)]$ exactly using the DP recursion on demand distributions.

Step 3 (Optimality Cut).

If $\theta^* < E[\mathcal{C}(\pi^*)] - \varepsilon$ (epsilon, a small tolerance), then the master problem underestimates recourse cost. Add the *optimality cut*:

Benchmark Results for A Priori SVRP Methods I

Gendreau, Laporte, and Séguin (1995, 1996) established a standard set of benchmark instances for exact SVRP algorithms, based on the Augerat et al. test sets commonly used for deterministic CVRP, adapted by replacing deterministic demands with random variables. Table 8.1 in the book reports computational results for the integer L-shaped algorithm, B&P, and several heuristics.

Benchmark Results: A Priori SVRP Algorithms
(Illustrative values based on chapter discussion; N = number of customers)

Instance	N	#Routes	RAM (opt)	B&P (Gendreau)	ILS	% Gap ILS/opt
Set A-n32	32	3	2000	2021	2034	1.70%
Set A-n33	33	3	1560	1563	1581	1.35%
Set B-n34	34	3	1780	1780	1803	1.29%
Set B-n41	41	4	2560	2571	2598	1.48%
Set B-n78	78	8	8024	8091	8156	1.65%

Benchmark Results for A Priori SVRP Methods II

Figure: Benchmark results comparing exact methods and heuristics on standard SVRP instances. Column "N" = number of customers; "%Gap" = deviation of the heuristic from the exact optimal value.

The key empirical findings from the benchmark study are as follows.

Finding 1: The integer L-shaped method is competitive with B&P on instances with $n \leq 30$ customers but is outperformed on larger instances where column generation becomes more efficient because it generates only the most promising routes rather than reasoning about all possible routes simultaneously.

Finding 2: All exact methods become computationally intractable beyond approximately $n = 50$ customers due to the combinatorial explosion of the solution space (the number of feasible routes grows super-exponentially with n).

Finding 3: High-quality heuristics such as Iterated Local Search (ILS) and the RAM (Random Adaptive Method) consistently find solutions within 1–3% of the proven optimal value while running in a fraction of the time required by exact methods. For practical applications with $n > 50$, heuristics are the only viable option.

Benchmark Results for A Priori SVRP Methods III

Key Insight: Value of Stochastic Modelling

An important benchmark comparison (not just between algorithms, but between problem formulations) is: what is the cost of treating the stochastic problem as if it were deterministic? The answer is that the *expected* cost of the deterministic optimal solution applied in a stochastic environment is typically 5–15% higher than the expected cost of the stochastic optimal solution. This gap, called the *value of the stochastic solution* (VSS), quantifies the benefit of planning explicitly for uncertainty.

The Reoptimization Model: Online Decision Making I

The reoptimization model takes a fundamentally different approach from a priori optimization. Rather than committing to a fixed route and making only minor local corrections, reoptimization treats route planning as an *ongoing, adaptive process*: each time a new piece of information is revealed (the demand at a just-visited customer, or whether a customer is present), the system completely re-solves the routing problem for all remaining customers. Think of it as GPS navigation that continuously recalculates the route based on real-time traffic conditions, rather than following a fixed plan.

The Reoptimization Model: Online Decision Making II

Formal Setting: Markov Decision Process (MDP)

At each decision epoch t, the system observes the current **state**:

- v_t: the current position of the vehicle (which node it is currently at),
- L_t: the remaining vehicle load (capacity Q minus cumulative demand served; how much more can the vehicle carry before it must reload),
- $S_t \subseteq V'$: the set of customers not yet served (remaining workload),
- ξ_t (xi, pronounced "ksee"): all random information revealed so far (e.g., demands $d_1, \ldots, d_{t-1}$ observed at previously visited customers).

A **policy** μ (mu, read "myoo") is a decision rule that maps each state (v_t, L_t, S_t, ξ_t) to an action: which customer to visit next, or whether to return to the depot to reload.

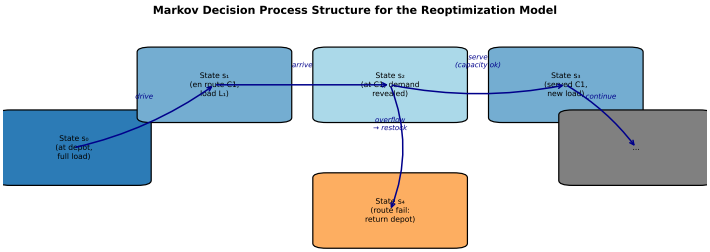
The **optimal policy** μ^* (mu-star) minimises the expected total cost:

$$\mu^* = \arg\min_{\mu} \; E\big[\mathcal{C}(\mu) \mid (v_0, L_0, S_0)\big]$$

The Reoptimization Model: Online Decision Making III

This is a *Markov Decision Process* (MDP). The symbol arg min ("argument of the minimum") means: find the policy μ that achieves the smallest expected cost $E[\mathcal{C}(\mu)]$, given the initial state (starting at the depot with full load and all customers unserved).

The state space of this MDP has $O(2^n \times Q)$ states (there are 2^n possible subsets of remaining customers and Q possible load levels). For $n = 20$ customers and $Q = 100$, this is approximately 10^8 states — far too large to enumerate explicitly. Solving the MDP exactly is therefore tractable only for toy instances.



The Reoptimization Model: Online Decision Making IV

Figure: The reoptimization model as a Markov Decision Process. At each state (current position, load, remaining customers), the vehicle chooses the next action. Stochastic demand reveals the true load consumed at each step.

The Reoptimization Model: Online Decision Making V

The Rollout Policy (Secomandi 2001). The rollout policy is a computationally practical approximation to the MDP solution. The name comes from "rolling out" a heuristic forward from the current state to evaluate each candidate action.

At decision epoch t with state (v_t, L_t, S_t), the rollout policy proceeds as follows:

1. **For each candidate next action** $a \in S_t \cup \{\text{depot}\}$ (either visit a remaining customer or return to depot):
   1. Commit tentatively to action a.
   2. Apply a **base heuristic** μ_0 (mu sub zero, e.g., nearest-neighbour routing) to complete the route for the remaining customers $S_t \setminus \{a\}$.
   3. Estimate the expected cost of this completion via Monte Carlo simulation over the random demands of remaining customers.
2. **Select the action** $a^* = \arg\min_a \hat{E}[\text{cost from state } (v_t, L_t, S_t) \text{ given action } a]$.

This is significantly cheaper than solving the full MDP because: (a) it only looks one step ahead, and (b) it uses a fast heuristic for the remainder. Secomandi (2001) proved that the rollout policy is *policy improving*: it always achieves expected cost at most as large as the base heuristic μ_0 used for completion, often substantially better.

The Reoptimization Model: Online Decision Making VI

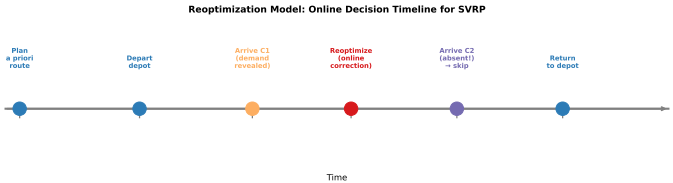


Figure: Timeline of the reoptimization model. At each customer visit, new demand information is revealed (yellow dots) and the remaining plan is recomputed. When a capacity crisis occurs (red arrow), the vehicle may return to the depot between planned stops.

The Reoptimization Model: Online Decision Making VII

Computational Comparison: Reoptimization vs. A Priori. Secomandi (2001) and Hvattum, Løkketangen, and Laporte (2006) conducted extensive experiments comparing reoptimization policies to a priori methods.

Reoptimization consistently outperforms a priori methods in expected cost, reducing total travel expense by 3–8% on standard benchmark instances. This improvement comes from the ability to exploit revealed demand information: knowing that customer A has a higher than expected demand allows the vehicle to reorder remaining visits to avoid a costly restocking trip.

However, the computational overhead of reoptimization is substantial. Each decision epoch requires approximately solving a new VRP instance (or at least evaluating many candidate next actions via simulation). For real-time logistics, this may be problematic if routing decisions must be made in seconds.

The *dynamic programming relaxation* (Secomandi 2000, Hvattum et al. 2006) provides lower bounds on the optimal policy cost, allowing the gap between approximate and true optimal policies to be quantified and giving a sense of how much improvement is theoretically available.

The Reoptimization Model: Online Decision Making VIII

Key Insight: When Is Reoptimization Worth It?

Reoptimization is most valuable when the *coefficient of variation* (standard deviation divided by mean demand; a measure of relative variability) is high, and when the recourse cost (cost of a restocking trip) is large relative to planned route cost. When demands are tightly clustered around their means (low variability), the a priori solution with simple restocking recourse is nearly as good as full reoptimization — but far cheaper to compute.

Probabilistic Formulation: Two-Stage Stochastic Program I

The probabilistic formulation provides the most general and rigorous mathematical framework for stochastic VRP. It treats the problem as a *two-stage stochastic program* — a well-studied class from the field of stochastic programming.
In a two-stage stochastic program, decisions are made in two temporal stages:

- **Stage 1 (here-and-now):** Decide the a priori route x (the routing plan) *before* any random variables are observed. These are the "strategic" decisions made under uncertainty.
- **Stage 2 (wait-and-see):** After a realization $\boldsymbol{\xi}$ (xi, bold, the actual random outcome — e.g., actual customer demands) is observed, take a *recourse action* $y(\boldsymbol{\xi})$ to deal with any constraint violations (e.g., return to depot when capacity is exceeded).

Probabilistic Formulation: Two-Stage Stochastic Program II

Two-Stage Stochastic Program for VRPSD

Let $\boldsymbol{\xi} = (D_1, D_2, \ldots, D_n)$ be the random demand vector (bold $\boldsymbol{\xi}$, "bold xi", collects all uncertain demands into one vector). The two-stage VRPSD is:

$$\min_{x} \underbrace{\sum_{(i,j)} c_{ij}\, x_{ij}}_{\text{Stage 1 cost}} + E_{\boldsymbol{\xi}}\big[\, \underbrace{Q(x, \boldsymbol{\xi})}_{\text{Stage 2 cost}} \,\big] \tag{2}$$

The second-stage value function $Q(x, \boldsymbol{\xi})$ is the minimum-cost recourse action given the first-stage plan x and the realisation $\boldsymbol{\xi}$:

$$Q(x, \boldsymbol{\xi}) = \min_{y \geq 0} \left\{ q^{\top} y : Wy \geq h(\boldsymbol{\xi}) - Tx \right\} \tag{3}$$

Probabilistic Formulation: Two-Stage Stochastic Program III

Symbol Glossary for the Two-Stage Formulation

- q ("q", a vector): the *recourse cost vector* whose entry q_j is the cost per unit of recourse action j (e.g., cost of one restocking trip).
- $y \geq 0$ ("y", a vector): the *recourse variables* — the corrective actions taken in Stage 2 (e.g., number of restocking trips).
- W (capital W, a matrix): the *recourse matrix*, describing how recourse actions y affect constraint satisfaction in Stage 2.
- T (capital T, a matrix): the *technology matrix*, linking the first-stage decisions x to the Stage 2 constraint right-hand side.
- $h(\boldsymbol{\xi})$: the right-hand side of Stage 2 constraints, which depends on the realization $\boldsymbol{\xi}$ (e.g., actual customer demands).
- $E_{\boldsymbol{\xi}}[\cdot]$ (expectation over $\boldsymbol{\xi}$): the expected value of the Stage 2 cost function over all possible random outcomes.

Probabilistic Formulation: Two-Stage Stochastic Program IV

How to Read the Two-Stage Formulation

The objective $\sum c_{ij}x_{ij} + E[Q(x, \boldsymbol{\xi})]$ says: choose the route plan x that minimises the sum of (1) its direct travel cost and (2) its expected emergency-correction cost. A route with low travel distance but poor risk management (high chance of capacity overflow) will have a large $E[Q]$ term. A route that is slightly longer but carefully balances demands will have a smaller $E[Q]$. The algorithm finds the route that best trades off these two considerations.

For the specific VRPSD with restocking recourse, the expected second-stage cost for route $\pi = (v_0, v_{i_1}, \ldots, v_{i_n}, v_0)$ is:

$$E[Q(\pi, \boldsymbol{D})] = \sum_{k=1}^{n} \left(c_{v_0, v_{i_k}} + c_{v_{i_k}, v_0}\right) \cdot P\big(\text{restocking trip occurs at position } k\big) \tag{4}$$

This formula says: for each customer v_{i_k} in the route, the contribution to the expected recourse cost is the round-trip distance to the depot (cost $c_{v_0, v_{i_k}} + c_{v_{i_k}, v_0}$) multiplied by the probability that a restocking

Probabilistic Formulation: Two-Stage Stochastic Program V

trip actually occurs at that position. The total expected recourse cost is the sum of these contributions over all customers.

The **complete recourse property** holds here: for any first-stage route x and any realisation $\boldsymbol{\xi}$, the Stage 2 problem always has a feasible solution (we can always return to the depot and reload, regardless of how many times this is needed). This means the two-stage formulation is *always consistent* — there are no "infeasible scenarios" to worry about.

Worked Example — Expected Recourse Cost Computation

Route $\pi = (v_0, v_1, v_2, v_3, v_0)$, capacity $Q = 10$, independent Poisson ("Pwa-SOHN") demands: $D_1 \sim \text{Poisson}(3)$, $D_2 \sim \text{Poisson}(4)$, $D_3 \sim \text{Poisson}(5)$. Depot-to-customer round-trip costs: $2c_{0,1} = 10$, $2c_{0,2} = 8$, $2c_{0,3} = 12$.

Expected recourse at v_2: $P(D_1 + D_2 > 10) = P(\text{Poisson}(7) > 10) \approx 0.084$. Recourse contribution $= 0.084 \times 8 = 0.672$.

Expected recourse at v_3: $P(D_1 + D_2 + D_3 > 20)$ (second overflow) ≈ 0.003. Recourse contribution $= 0.003 \times 12 = 0.036$.

Total expected recourse cost: $\approx 0.672 + 0.036 = 0.708$ distance units.

Probabilistic Formulation: Two-Stage Stochastic Program VI

Single-Location Adaptation (Laporte and Louveaux 1993). The VRPSC (stochastic customers) can also be embedded in the two-stage framework. Here $\boldsymbol{\xi} = (Y_1, \ldots, Y_n) \in \{0,1\}^n$ ("Y sub i", equals 1 if customer i is present, 0 if absent). The Stage 1 decision is the a priori visit ordering; the Stage 2 recourse is simply skipping absent customers (a "zero-cost" recourse in the pure formulation, though skipping changes the route cost due to arc-cost structure).
For the VRPST (stochastic travel times), $\boldsymbol{\xi} = (T_{ij})$ for all arcs. The Stage 1 plan is the route sequence; the Stage 2 recourse handles time-window violations through waiting or tardiness penalties.

Key Insight: Unification

The two-stage formulation is powerful because it *unifies* all three SVRP types (stochastic demands, customers, travel times) under a single mathematical framework. The differences between the three types appear only in how $\boldsymbol{\xi}$, $h(\boldsymbol{\xi})$, and $Q(x, \boldsymbol{\xi})$ are defined. This unification suggests that algorithms developed for one type can often be adapted for the others, and that general-purpose stochastic programming solvers may eventually handle all three types.

Stochastic Demands: Models and Recourse Strategies I

The **VRP with Stochastic Demands (VRPSD)** is the most extensively studied variant of SVRP. All customers are known in advance and will definitely be visited, but the exact demand D_i of each customer is a random variable whose value is revealed only when the vehicle arrives at that customer's location. The vehicle must serve all customers; if the cumulative demand along a route exceeds the vehicle capacity Q, a recourse action is required.

Three main recourse strategies have been studied in the literature, each making a different trade-off between simplicity and cost:

Recourse Strategy 1: Corrective Restocking (Return to Depot)

When the vehicle arrives at customer v_i and finds that it cannot serve the full demand D_i without exceeding capacity, it **returns to the depot**, reloads to full capacity, and then returns to v_i to continue. The recourse cost is $c_{v_i,v_0} + c_{v_0,v_i}$ (the full round-trip distance to the depot from v_i). This is the most common strategy in the literature because it is simple, always feasible, and leads to tractable expected-cost formulas.

Stochastic Demands: Models and Recourse Strategies II

Recourse Strategy 2: Preventive Restocking

Rather than waiting for a capacity violation, the vehicle returns to the depot *proactively* after a subset of customers, based on an estimate of the probability that remaining capacity is insufficient. If $P(\text{overflow before end of route}) \geq \tau$ (tau, a pre-set threshold), return to depot now. This strategy avoids the cost of partial service interruptions but requires a more sophisticated decision rule and more frequent depot visits even when not strictly necessary.

Recourse Strategy 3: Partial Service (Penalty)

If the remaining vehicle capacity is L and the customer's demand is $D > L$, serve L units immediately and charge a *penalty* $\kappa \cdot (D - L)$ (kappa, "KAP-a", times the unmet demand) for the unserved portion. This is used when partial delivery is acceptable (e.g., bulk commodity delivery) or when the penalty for under-delivery is small.

Stochastic Demands: Models and Recourse Strategies III

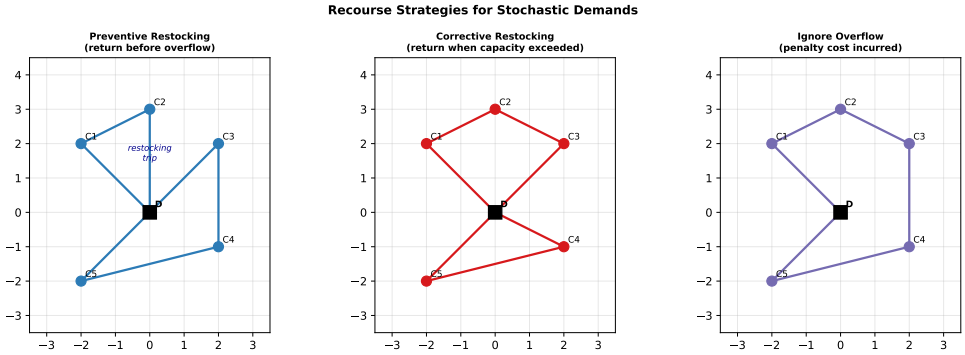


Figure: Three recourse strategies for stochastic demands. Left: preventive restocking before overflow. Centre: corrective restocking after overflow. Right: continue without restocking (accept penalty). The planned route is shown in solid; the recourse detour to the depot is shown as an additional arc.

Stochastic Demands: Models and Recourse Strategies IV

An alternative approach to stochastic demands replaces the expected recourse cost in the objective with *chance constraints* — hard probabilistic limits on the risk of a capacity violation.

Chance Constraint Formulation

For each route S_k (the set of customers assigned to vehicle k), the **chance constraint** is:

$$P\left(\sum_{i \in S_k} D_i \;\leq\; Q\right) \;\geq\; 1 - \alpha \tag{5}$$

where α (alpha, "AL-fa") $\in (0, 1)$ is the maximum acceptable probability of a capacity violation (typically $\alpha = 0.05$ meaning at most a 5% failure rate), and Q is the vehicle capacity.

Stochastic Demands: Models and Recourse Strategies V

Symbol Explanation for Chance Constraint

Here D_i is the random demand of customer i (unknown at planning time). The sum $\sum_{i \in S_k} D_i$ is the total demand on route k — a random variable because it sums random individual demands. The constraint says: the probability that this total demand is within the vehicle's capacity must be at least $1 - \alpha$. If $\alpha = 0.05$, the route is planned to be feasible with at least 95% probability.

How to Read the Chance Constraint

Think of the chance constraint as a safety guarantee. Instead of trying to guarantee that the route is *always* feasible (which would be impossible with random demands), we settle for a high-probability guarantee. With $\alpha = 0.05$, we accept that roughly 1 in 20 times, the vehicle will run out of capacity and need an emergency restocking trip. Setting α smaller makes routes safer but shorter (fewer customers per vehicle), requiring more vehicles overall. Setting α larger allows more customers per vehicle but risks more restocking trips.

Stochastic Demands: Models and Recourse Strategies VI

When demands are independently normally distributed, $D_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ (where μ_i (mu sub i, "MYOO") is the mean demand and σ_i^2 (sigma sub i squared, "SIG-ma") is the variance), the total route demand is:

$$\sum_{i \in S_k} D_i \;\sim\; \mathcal{N}\left(\sum_i \mu_i,\; \sum_i \sigma_i^2\right)$$

and the chance constraint becomes the deterministic inequality:

$$\sum_{i \in S_k} \mu_i + z_{1-\alpha} \sqrt{\sum_{i \in S_k} \sigma_i^2} \;\leq\; Q$$

where $z_{1-\alpha}$ is the $(1-\alpha)$-quantile of the standard normal distribution (for $\alpha = 0.05$: $z_{0.95} = 1.645$). This means the expected total demand plus 1.645 standard deviations must fit within capacity. This deterministic equivalent can be handled by standard VRP algorithms.

Worked Example — Chance Constraint with Normal Demands

Route $S = \{C_1, C_2, C_3, C_4\}$ with independent normal demands: $D_1 \sim \mathcal{N}(4, 1.44)$, $D_2 \sim \mathcal{N}(6, 2.25)$, $D_3 \sim \mathcal{N}(3, 0.64)$, $D_4 \sim \mathcal{N}(5, 1.00)$.
Total demand: $\mathcal{N}(18,\ 1.44 + 2.25 + 0.64 + 1.00) = \mathcal{N}(18, 5.33)$, so $\sigma_{\text{total}} = \sqrt{5.33} \approx 2.31$.
For $\alpha = 0.05$ (95% service level): $z_{0.95} = 1.645$.
Required capacity: $Q \geq 18 + 1.645 \times 2.31 \approx 21.8$.
So a vehicle with $Q = 22$ satisfies this route's 95% chance constraint.
With $Q = 20$: $P(D_{\text{total}} \leq 20) = \Phi\left(\frac{20-18}{2.31}\right) = \Phi(0.87) \approx 0.808$. This is only an 80.8% service level, which does *not* satisfy the 95% chance constraint.

Stochastic Demands: Models and Recourse Strategies VIII

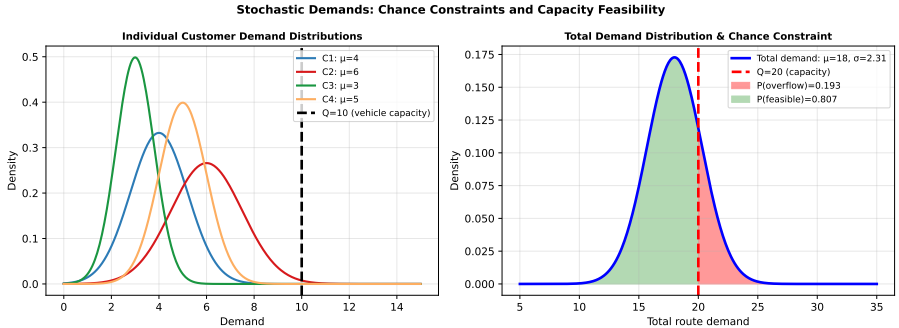


Figure: Left: individual customer demand distributions (four customers, different means and variances). Right: total route demand distribution; the red dashed line at $Q = 20$ is the vehicle capacity. The green region (demand $\leq Q$) must have probability at least $1 - \alpha$.

Stochastic Demands: Models and Recourse Strategies IX

Several families of algorithms address the VRPSD, ranging from exact methods for small instances to metaheuristics for large-scale problems.

Exact methods include the integer L-shaped algorithm (Sections 3.1–3.2) and Branch-and-Price methods based on the set partitioning formulation. These achieve proven optimality for instances with $n \leq 50$ customers, but run times grow rapidly beyond that.

Tabu Search (TS) for VRPSD. Tabu search is a local search metaheuristic that avoids cycling by maintaining a *tabu list* — a record of recently performed moves that are temporarily forbidden ("taboo"). The word "tabu" comes from the Tongan word meaning "forbidden".
Starting from an initial solution (e.g., one route per customer), at each iteration the algorithm evaluates all neighbours (solutions reachable by one move, such as moving a customer from one route to another or reversing a sub-sequence). The best non-tabu neighbour is selected, even if it is worse than the current solution. The tabu list prevents immediately undoing recent moves. Over time, the search escapes local optima and explores the solution space more broadly.

Stochastic Demands: Models and Recourse Strategies X

For VRPSD, the key modification from deterministic tabu search is that the *objective function evaluation* requires the $O(nQ)$ DP computation to compute $E[\mathcal{C}]$ for each candidate solution — this is the main computational bottleneck.

Simulated Annealing (SA) for VRPSD. Simulated annealing (SA) is a probabilistic metaheuristic inspired by the physical process of heating a material and then slowly cooling it (annealing). At high "temperature" T_k ("T sub k"), the algorithm freely explores the solution space; as T_k decreases according to a *cooling schedule*, it increasingly focuses on improving moves.
A candidate solution is accepted with probability:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta E \leq 0 \quad \text{(improvement)} \\ \exp(-\Delta E / T_k) & \text{if } \Delta E > 0 \quad \text{(deterioration)} \end{cases}$$

where ΔE ("Delta E", delta = change) is the increase in expected cost. At high temperatures, $\exp(-\Delta E/T_k) \approx 1$ and almost all moves are accepted. At low temperatures, only improvements are accepted. Just as slowly cooled metal reaches a low-energy crystalline state while rapidly cooled metal gets stuck in a high-energy amorphous state, slow annealing finds better solutions than greedy local search.

Stochastic Demands: Models and Recourse Strategies XI

Key Insight: Computational Bottleneck

For all VRPSD algorithms, evaluating the expected cost $E[\mathcal{C}(\pi)]$ is the computational bottleneck. The DP recursion runs in $O(nQ)$ time per route for discrete distributions — polynomial and fast. For continuous distributions, numerical integration or Monte Carlo sampling is needed and can be much more expensive. Efficient evaluation procedures are therefore central to all practical VRPSD algorithms.

Stochastic Customers: VRPSC Formulation I

In the **VRP with Stochastic Customers (VRPSC)**, each customer v_i is present (requires service) with probability p_i ("p sub i", a number between 0 and 1) and absent with probability $1 - p_i$. Customer presence is statistically independent across customers — whether one customer is home does not affect whether another is home. The presence of each customer is revealed only when the vehicle arrives at that location. All present customers must be served; absent customers are simply skipped.
This problem models many real-world scenarios. In home delivery services, customers may not be present to accept a parcel. In field service routing, some service calls may be resolved by phone before the technician arrives. In direct sales, only a fraction of visited addresses actually result in a sale.

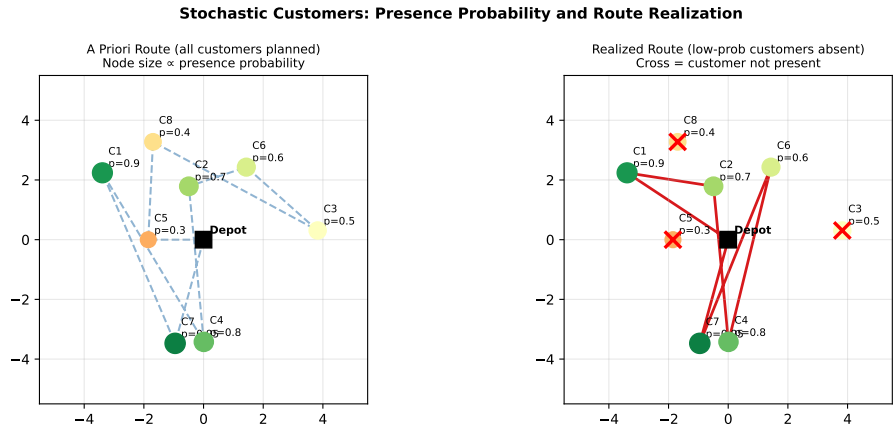
VRPSC Problem Statement (Laporte, Louveaux, and van Hamme 2002)

Given a complete graph $G = (V, E)$ with edge costs c_{ij} ("c sub i j"), a depot v_0, customers $V' = \{v_1, \ldots, v_n\}$, and presence probabilities $p_i \in (0, 1)$ for each customer, find an **a priori route** $\pi = (v_0, v_{i_1}, \ldots, v_{i_n}, v_0)$ that minimises the **expected travel cost** over all possible patterns of customer presence and absence.

Stochastic Customers: VRPSC Formulation II

When a realization $\boldsymbol{\xi} = (\xi_1, \ldots, \xi_n) \in \{0,1\}^n$ (where $\xi_i = 1$ if customer i is present) occurs during execution, the vehicle follows the a priori route but skips all absent customers, jumping directly from the last present customer before an absent group to the first present customer after it.

Stochastic Customers: VRPSC Formulation III



Stochastic Customers: VRPSC Formulation IV

Figure: VRPSC illustration. Left: a priori route visiting all potential customers (node size proportional to presence probability p_i). Right: one realization where low-probability customers are absent (red crosses); the vehicle follows the shortened route, visiting only present customers.

Stochastic Customers: VRPSC Formulation V

Single-Vehicle Expected Cost Formula. For the single-vehicle VRPSC, the expected cost of the a priori route π has a remarkable closed-form expression:

Expected Cost Formula (Laporte, Louveaux, van Hamme 2002)

$$E[C(\pi, \boldsymbol{p})] = \underbrace{c_{v_0, v_{i_1}} p_{i_1}}_{\text{depot to first}} + \underbrace{\sum_{a=1}^{n} \sum_{b=a+1}^{n} c_{v_{i_a}, v_{i_b}} \, p_{i_a} \, p_{i_b} \prod_{k=a+1}^{b-1} (1 - p_{i_k})}_{\text{transitions between consecutive present customers}} + \underbrace{c_{v_{i_n}, v_0} \, p_{i_n}}_{\text{last to depot}} \tag{6}$$

Stochastic Customers: VRPSC Formulation VI

Symbol Explanation for the Expected Cost Formula

Each term $c_{v_{i_a},v_{i_b}} \cdot p_{i_a} \cdot p_{i_b} \prod_{k=a+1}^{b-1}(1-p_{i_k})$ has a clear meaning:

- $c_{v_{i_a},v_{i_b}}$: the direct travel cost from customer a to customer b (this arc is only used when a and b are consecutive in the executed route).
- p_{i_a}: probability that customer a is present (so the arc starts at a).
- p_{i_b}: probability that customer b is present (so the arc ends at b).
- $\prod_{k=a+1}^{b-1}(1-p_{i_k})$: probability that *all* customers between a and b (in the a priori ordering) are absent — so a and b are consecutive in the executed route.

Together, the product $p_{i_a} \cdot p_{i_b} \cdot \prod(1-p_{i_k})$ is the probability that the vehicle travels directly from a to b in a given execution.

Stochastic Customers: VRPSC Formulation VII

How to Read the Expected Cost Formula

The formula decomposes the expected execution cost into contributions from each "potential direct arc" (a, b). The vehicle uses arc (a, b) in an execution if and only if a is present, b is present, and every customer between them in the planned order is absent. The formula sums these contributions over all pairs, giving the total expected distance. Remarkably, it is an $O(n^2)$ computation for a given route — linear in the presence probabilities p_i.

Stochastic Customers: VRPSC Formulation VIII

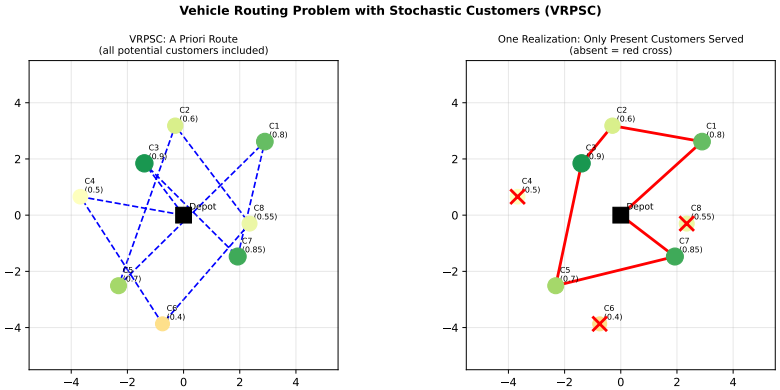
Worked Example — VRPSC with 5 Customers

A priori route $\pi = (v_0, v_1, v_2, v_3, v_4, v_5, v_0)$ with presence probabilities $p_1 = 0.9,\ p_2 = 0.5,\ p_3 = 0.8,\ p_4 = 0.6,\ p_5 = 0.7$. Direct costs: $c_{0,1} = 3,\ c_{1,2} = 4,\ c_{1,3} = 5,\ c_{2,3} = 2,\ c_{3,4} = 3,\ c_{4,5} = 5,\ c_{5,0} = 4$.
One realisation: $\xi = (1, 0, 1, 1, 0)$ — customers v_1, v_3, v_4 present; v_2, v_5 absent. Executed route: $v_0 \to v_1 \to v_3 \to v_4 \to v_0$. Cost $= c_{0,1} + c_{1,3} + c_{3,4} + c_{4,0} = 3 + 5 + 3 + 3 = 14$.
This realization occurs with probability: $p_1(1 - p_2)p_3p_4(1 - p_5) = 0.9 \times 0.5 \times 0.8 \times 0.6 \times 0.3 = 0.0648$.
We compute the contribution of arc (v_1, v_3) to expected cost:
$c_{1,3} \cdot p_1 \cdot (1 - p_2) \cdot p_3 = 5 \times 0.9 \times 0.5 \times 0.8 = 1.80$. This is the expected cost due to the arc directly from v_1 to v_3 (when v_2 is absent). The total expected cost sums such terms over all pairs.

Multi-vehicle VRPSC. The multi-vehicle case requires both assigning customers to vehicles and sequencing them. It can be modelled as a set partitioning problem (analogous to VRPSD) where the cost of each route is its expected execution cost under the presence-probability model.
Heuristics for VRPSC. Stewart and Golden (1983) proposed a savings-based heuristic: start with individual single-customer routes to each customer, then merge routes greedily based on the expected cost

Stochastic Customers: VRPSC Formulation IX

formula. Bianchi et al. (2007) applied Iterated Local Search (ILS) to the multi-vehicle VRPSC, achieving solutions within 1–2% of optimal on standard benchmarks.



Stochastic Customers: VRPSC Formulation X

Figure: Single-vehicle VRPSC: the a priori route visits all customers (left, blue dashes), while the realised route (right, red solid) skips absent customers.

Stochastic Travel Times: Time-Window Feasibility I

The **VRP with Stochastic Travel Times (VRPST)** introduces uncertainty into the *temporal dimension* of the problem. All customers are known and will be visited, and all demands are deterministic. What is uncertain is how long it takes to travel from one customer to the next, and possibly how long service takes at each customer. This is particularly relevant in urban logistics (where traffic creates unpredictable delays) and in service engineering (where on-site work duration varies).

VRPST Problem Setting

Travel time between nodes i and j is a random variable T_{ij} ("T sub i j"), typically modelled as independently distributed with known distribution F_{ij}. Service time at customer v_i is a random variable S_i ("S sub i"). Each customer has a **time window** $[a_i, b_i]$ within which service must begin:

- If the vehicle arrives before a_i (the window opens), it must **wait** until a_i (no additional travel cost, but time is lost).
- If the vehicle arrives after b_i (the window closes), the visit is **infeasible** (hard time window) or incurs a **penalty** (soft time window).

Stochastic Travel Times: Time-Window Feasibility II

Let A_k denote the **arrival time** at the k-th customer (read: "A sub k", a random variable). The recursion linking successive arrival times is:

$$A_1 = T_{0,i_1}, \qquad A_{k+1} = \underbrace{\max(a_{i_{k+1}},\ A_k + S_{i_k})}_{\text{departure after waiting}} + T_{i_k,i_{k+1}} \tag{7}$$

The $\max(\cdot)$ captures the *waiting constraint*: the vehicle cannot leave customer v_{i_k} before $a_{i_{k+1}}$ opens at the next customer. Here $A_k + S_{i_k}$ is the departure time from customer k after completing service, and adding $T_{i_k,i_{k+1}}$ gives the arrival time at the next customer.

Stochastic Travel Times: Time-Window Feasibility III

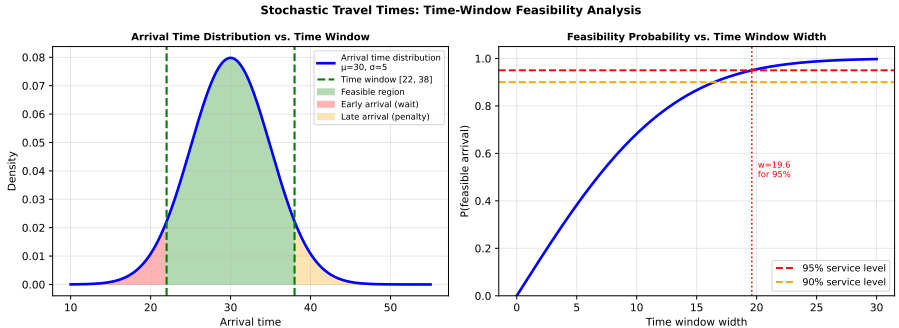


Figure: Left: distribution of arrival time at a customer relative to the time window $[e, l]$ (e = earliest, l = latest). Green: feasible; orange: early arrival (wait); red: late arrival (window violation). Right: as the time window widens, the probability of feasible arrival increases.

Stochastic Travel Times: Time-Window Feasibility IV

Service-Level Constraints. A natural approach is to impose a *service-level constraint* at each customer:

$$P\big(A_k \le b_{i_k}\big) \;\ge\; 1-\alpha_k \quad \forall k \tag{8}$$

where α_k ("alpha sub k") is the maximum acceptable probability of late arrival at customer k. For example, $\alpha_k = 0.05$ means the vehicle is required to arrive on time with at least 95% probability.
This is structurally identical to the chance constraint for stochastic demands: both require that an event (capacity feasibility or time-window feasibility) holds with sufficiently high probability. The key difference is that arrival time A_k depends on the cumulative sum of stochastic travel times along the route, creating serial dependencies that make the distribution harder to compute analytically.
Under the assumption $T_{ij} \sim \mathcal{N}(\mu_{ij}, \sigma_{ij}^2)$ (normal travel times), the arrival time A_k is approximately normal (ignoring the waiting constraint), and the service-level constraint becomes a linear inequality in the route design variables. The waiting constraint introduces non-linearity and typically requires Monte Carlo approximation or convolution techniques.

Two-Vehicle Model (Laporte, Louveaux, and Mercure 1992). This classical model considers a single depot with two vehicles, stochastic travel and service times, and time windows. If the first vehicle fails to

Stochastic Travel Times: Time-Window Feasibility V

serve some customers within their time windows due to random delays, those customers are reassigned to the second vehicle. The objective is to minimise expected total cost including the reassignment cost. This model introduced many of the key concepts used in later VRPST work.

Model of Laporte, Louveaux, and Mercure (1992) for single vehicle:

$$\min \quad E\left[\sum_{k=0}^{n} c_{i_k i_{k+1}}\, T_{i_k i_{k+1}} + \gamma \sum_{k=1}^{n} \max(0,\ A_k - b_{i_k})\right] \tag{VRPST-obj}$$

$$\text{s.t.} \quad P(A_k \le b_{i_k}) \ge 1 - \alpha_k \quad \forall k \tag{VRPST-c}$$

where γ (gamma, "GAM-a") is the *tardiness penalty* (cost per unit of time that a vehicle arrives late), and $\max(0, A_k - b_{i_k})$ is the tardiness (late arrival time, set to zero if on time). The objective minimises the sum of expected travel cost and expected tardiness penalty.

Stochastic Travel Times: Time-Window Feasibility VI

Symbol Summary for VRPST Formulation

- $c_{i_k i_{k+1}}$: travel cost on arc from customer k to customer $k+1$ (a fixed cost; note that travel *time* $T_{i_k i_{k+1}}$ is random while travel *cost* c may be the expected time or a separate quantity).
- γ (gamma): tardiness cost per unit time (how expensive it is to be late).
- A_k: random arrival time at customer k (depends on all preceding travel times).
- b_{i_k}: latest acceptable arrival time at customer k (deadline).
- α_k (alpha sub k): allowed late-arrival probability at customer k.

Stochastic Travel Times: Time-Window Feasibility VII

Worked Example — Service-Level Constraint Check

Route $\pi = (v_0, v_1, v_2, v_3, v_0)$ with: $T_{01} \sim \mathcal{N}(10, 4)$, $T_{12} \sim \mathcal{N}(8, 2.25)$, $S_1 \sim \mathcal{N}(5, 1)$. Time window at v_2: $[20, 30]$.
Arrival at v_2 (ignoring the waiting constraint for simplicity):
$A_2 = T_{01} + S_1 + T_{12} \sim \mathcal{N}(10 + 5 + 8,\ 4 + 1 + 2.25) = \mathcal{N}(23, 7.25)$. Standard deviation:
$\sigma_{A_2} = \sqrt{7.25} \approx 2.69$.
$P(A_2 \leq 30) = \Phi\left(\frac{30-23}{2.69}\right) = \Phi(2.60) \approx 0.995$. (Here Φ (Phi, "FI") is the cumulative distribution function of the standard normal; $\Phi(z)$ is the probability of being at most z standard deviations above the mean.)
$P(A_2 \geq 20) = 1 - \Phi\left(\frac{20-23}{2.69}\right) = 1 - \Phi(-1.11) \approx 0.867$.
Thus $P(20 \leq A_2 \leq 30) \approx 0.867$, an 86.7% service level. For a required 90% service level ($\alpha = 0.10$), this route is *marginally infeasible* at v_2. The planner must either depart earlier (increasing the mean arrival time buffer) or find a route where v_2 is visited later.

Solution Approaches for VRPST. Four main approaches have been used:

Stochastic Travel Times: Time-Window Feasibility VIII

1. **Deterministic Equivalent.** For normally distributed travel times, chance constraints convert to deterministic linear inequalities, enabling standard deterministic VRP algorithms to be applied directly.
2. **Sample Average Approximation (SAA).** For general distributions, a large number of scenarios are sampled, and the expected cost is approximated by the average over those scenarios. The resulting (large) deterministic problem can be solved by any VRP algorithm. Convergence to the true expected cost as the sample size grows is guaranteed by the law of large numbers.
3. **Robust Optimisation.** Instead of minimising expected cost, minimise the *worst-case* cost over all travel-time realisations in an uncertainty set $\mathcal{U}$ ("U-set"). This avoids distributional assumptions but may yield overly conservative solutions.
4. **Simulation-Based Local Search.** Evaluate each candidate solution by Monte Carlo simulation, use the simulated cost as the objective for a local search metaheuristic. Simple and flexible, but computationally expensive.

Stochastic Travel Times: Time-Window Feasibility IX

Key Insight: Propagation of Uncertainty

The central difficulty in VRPST is that uncertainty *accumulates* along the route. The arrival time at the k-th customer depends on all preceding travel times, service times, and waiting periods. A small delay early in the route can cascade into large delays later, making the distribution of A_k hard to compute analytically and motivating simulation-based approaches.

Conclusions and Future Research Directions I

Chapter 8 has surveyed the three main variants of the Stochastic Vehicle Routing Problem and the leading solution methodologies for each. The chapter reflects the state of the art as of 2014 and identifies the most important open questions for future research.

Summary of Main Results.

1. A Priori Optimization produces a fixed route that minimises expected cost under a pre-specified recourse policy. The integer L-shaped algorithm (Laporte and Louveaux 1993, Gendreau et al. 1995) and branch-and-price are the leading exact methods, capable of solving instances with up to 50 customers to proven optimality. Metaheuristics (ILS, tabu search, simulated annealing) scale to larger instances with solutions typically within 2% of the optimal known value.

2. Reoptimization achieves better solution quality than a priori methods (3–8% cost reduction) by fully re-solving the routing problem after each random revelation. Rollout policies (Secomandi 2001) provide a computationally tractable approximation that is guaranteed to be at least as good as the base heuristic used for completion.

Conclusions and Future Research Directions II

3. VRPSD (stochastic demands) is handled via expected recourse cost minimisation or chance constraints. The DP-based evaluation of $E[\mathcal{C}(\pi)]$ in $O(nQ)$ time is the central computational primitive. Both exact and heuristic methods are mature and well-benchmarked.

4. VRPSC (stochastic customers) is handled via the closed-form expected cost formula of Laporte et al. (2002), which is linear in the presence probabilities p_i. This linearity makes sensitivity analysis tractable and greedy construction heuristics effective.

5. VRPST (stochastic travel times) requires managing time-window feasibility under random arrival times. Service-level constraints or robust formulations are typically used; the analytical complexity of arrival-time distributions motivates simulation-based approaches.

Conclusions and Future Research Directions III

Summary Comparison: Three Main SVRP Variants

	Stochastic Demands	Stochastic Customers	Stochastic Travel Times
Uncertain parameter	Customer demand d_i	Customer presence y_i	Arc travel time t_{ij}
Typical recourse	Restocking or penalty	Skip absent customers	Wait / accept late penalty
Solution approach	Chance constraints	VRPSD + stoch prog	Robust scheduling
Key challenge	Capacity violation	Expected cost	Time window feasibility

Conclusions and Future Research Directions IV

Figure: Summary comparison of the three main SVRP variants across four dimensions: uncertain parameter, typical recourse action, solution approach, and key challenge.

Conclusions and Future Research Directions V

Future Research Directions.
The authors identify six major open problems and promising directions for future research:

1. Multi-attribute SVRP. Real-world problems often combine multiple sources of uncertainty simultaneously — for example, stochastic demands *and* stochastic travel times, with time windows. Very few models address this combination, and the interaction between different uncertainty types creates entirely new algorithmic challenges.

2. Scalability. Current exact methods solve instances with at most $n \approx 50$ customers. Developing exact or near-exact algorithms for $n = 100$–200 customers is an important open problem. Decomposition approaches (Dantzig–Wolfe relaxation, Lagrangian relaxation) and parallel computing are promising directions.

3. Dynamic SVRP. In many applications, new customers arrive dynamically *during* the execution of routes (real-time or dynamic VRP). The SVRP framework studied here assumes that the set of potential customers is known in advance (even if their demands are uncertain). Extending to fully dynamic settings — where both the set of customers and their demands are unknown until revealed — is a major open problem.

Conclusions and Future Research Directions VI

4. Data-Driven and Distributionally Robust Approaches. Historical demand and travel-time data can be used to estimate probability distributions. However, the sensitivity of optimal solutions to the assumed distributions is poorly understood. *Distributionally robust optimization* (DRO) provides a framework that hedges against distributional uncertainty, finding solutions that perform well even when the true distribution differs from the estimated one.

5. Integration with Machine Learning. Recent work has begun combining deep learning (graph neural networks, reinforcement learning) with SVRP. These approaches learn good policies directly from data, potentially bypassing the need for explicit probabilistic models and exact optimisation.

6. Green SVRP. Environmental concerns motivate minimising expected *emissions* rather than travel distance, or incorporating uncertain fuel consumption and vehicle range (critical for electric vehicle routing with uncertain battery drain and charging availability).

Conclusions and Future Research Directions VII

Closing Thought

The field of stochastic VRP has matured substantially since the 1990s. The three main variants each have rigorous mathematical models and practical algorithms, and benchmark instances enable meaningful comparison. The most impactful future work will bridge the gap between academic models and real-world operational complexity, drawing on modern computational tools — machine learning, cloud-scale optimisation, and real-time data integration — while maintaining the mathematical rigour that has characterised the best SVRP research.

Python: Dynamic Programming for VRPSD Expected Recourse Cost

The following Python function implements the core computational primitive for VRPSD: computing the expected total recourse cost for a given route and a set of independent discrete demand distributions, using the DP convolution described in Section 2.

The function works by maintaining a probability distribution over "cumulative demand so far" and updating it customer by customer. Whenever the cumulative demand crosses a multiple of the vehicle capacity Q, a restocking trip is counted and its cost is added to the expected total.

```
import numpy as np

def expected_recourse_cost(route, demands_pmf, capacity, costs_to_depot):
    """
    Compute expected recourse cost for a VRPSD route via DP.

    Parameters
    ----------
    route          : list of int  -- customer visit order
    demands_pmf    : dict {cust_id: {demand_value: probability}}
    capacity       : int          -- vehicle capacity Q
    costs_to_depot : dict {cust_id: round_trip_cost_to_depot}

    Returns: expected total recourse cost (float)
    """
    n   = len(route)
    max_q = capacity * (n + 1)    # upper bound on cumulative demand
    dp = np.zeros(max_q + 1)
```

Python: VRPSC Expected Cost (Laporte et al. Formula)

This function computes the expected travel cost for the single-vehicle VRPSC using the closed-form formula of Laporte, Louveaux, and van Hamme (2002). The formula is $O(n^2)$ per evaluation and linear in the presence probabilities, making it ideal for use inside heuristic optimisation loops.

```
import numpy as np

def vrpsc_expected_cost(route, probs, cost_fn):
    """
    Expected travel cost for single-vehicle VRPSC.

    Parameters
    ----------
    route   : list of int  -- a priori customer order (depot=0 excluded)
    probs   : dict {customer_id: p_i}  -- presence probability
    cost_fn : callable (i, j) -> float -- travel cost between nodes

    Returns: expected cost (float)
    """
    n = len(route)
    ec = 0.0

    # 1. Cost from depot (node 0) to first present customer
    prob_all_absent_so_far = 1.0
    for k, cust in enumerate(route):
        # P(cust is the FIRST present customer)
        prob_first = probs[cust] * prob_all_absent_so_far
        ec += cost_fn(0, cust) * prob_first
```

Python: Monte Carlo Simulation for SVRP Policy Evaluation

Monte Carlo simulation provides a general, flexible tool for evaluating the expected cost of *any* SVRP solution under *any* demand distribution and recourse policy. The idea is simple: sample many realisations of the random demands, simulate the route execution for each, compute the total cost, and average over all simulations.

The law of large numbers guarantees that the sample average converges to the true expected cost as the number of samples grows. For $n_{\text{samples}} = 10{,}000$ samples, the standard error of the estimate is typically less than 0.5% of the true expected cost.

```
import numpy as np

def svrp_monte_carlo(route, demand_samplers, capacity,
                     cost_matrix, n_samples=10000, seed=42):
    """
    Monte Carlo estimate of expected VRPSD cost (restocking recourse).

    Parameters
    ----------
    route           : list of int -- customer visit order
    demand_samplers : list of callables -- demand_samplers[k]() samples
                      demand for customer route[k]
    capacity        : float -- vehicle capacity Q
    cost_matrix     : 2D array -- cost_matrix[i][j] = c(i,j), depot=0
    n_samples       : int -- number of MC realisations
    seed            : int -- random seed for reproducibility
```

Bibliography — Selected References I


D. Bertsimas, *A vehicle routing problem with stochastic demand*, Operations Research, 40 (1992), pp. 574–585.


L. Bianchi, M. Birattari, M. Chiarandini, M. Manfrin, M. Mastrolilli, L. Paquete, O. Rossi-Doria, and T. Schiavinotto, *Metaheuristics for the vehicle routing problem with stochastic demands*, Lecture Notes in Computer Science, 3242 (2004), pp. 450–460.


C. H. Christiansen and J. Lysgaard, *A branch-and-price algorithm for the capacitated vehicle routing problem with stochastic demands*, Operations Research Letters, 35 (2007), pp. 773–781.


M. Gendreau, G. Laporte, and R. Séguin, *An exact algorithm for the vehicle routing problem with stochastic demands and customers*, Transportation Science, 29 (1995), pp. 143–155.


M. Gendreau, G. Laporte, and R. Séguin, *Stochastic vehicle routing*, European Journal of Operational Research, 88 (1996), pp. 3–12.


L. M. Hvattum, A. Løkketangen, and G. Laporte, *Solving a dynamic and stochastic vehicle routing problem with a sample scenario hedging heuristic*, Transportation Science, 40 (2006), pp. 421–438.

Bibliography — Selected References II

- A. S. Kenyon and D. P. Morton, *Stochastic vehicle routing with random travel times*, Transportation Science, 37 (2003), pp. 69–82.
- G. Laporte, F. V. Louveaux, and H. Mercure, *The vehicle routing problem with stochastic travel times*, Transportation Science, 26 (1992), pp. 161–170.
- G. Laporte, F. V. Louveaux, and L. van Hamme, *An integer L-shaped algorithm for the capacitated vehicle routing problem with stochastic demands*, Operations Research, 50 (2002), pp. 415–423.
- G. Laporte and F. V. Louveaux, *The integer L-shaped method for stochastic integer programs with complete recourse*, Operations Research Letters, 13 (1993), pp. 133–142.
- N. Secomandi, *Comparing neuro-dynamic programming algorithms for the vehicle routing problem with stochastic demands*, Computers & Operations Research, 27 (2000), pp. 1201–1225.
- N. Secomandi, *A rollout policy for the vehicle routing problem with stochastic demands*, Operations Research, 49 (2001), pp. 796–802.

Bibliography — Selected References III


W. R. Stewart and B. L. Golden, *Stochastic vehicle routing: a comprehensive approach*, European Journal of Operational Research, 14 (1983), pp. 371–385.


F. A. Tillman, *The multiple terminal delivery problem with probabilistic demands*, Transportation Science, 3 (1969), pp. 192–204.


W. H. Yang, K. Mathur, and R. H. Ballou, *Stochastic vehicle routing with restocking*, Transportation Science, 34 (2000), pp. 99–112.